

Security Vulnerability Audit Report

Application	Language	Vulnerabilities	Date
Login + File Manager CLI	Python 3.x	5 Found	11 Jun 2026

Prepared by: Ashu | CodeAlpha Cybersecurity Internship — Task 3
Methodology: Static Code Analysis + Manual Inspection

1. Executive Summary

This report presents the findings of a secure code review conducted on a Python-based Login and File Manager CLI application. The audit was performed as part of the CodeAlpha Cybersecurity Internship (Task 3) using static analysis and manual code inspection techniques.

The review identified **five (5) security vulnerabilities** across two source files. These range from critical authentication bypass flaws to medium-severity information disclosure issues. Each finding is documented with a description, proof-of-concept attack vector, severity rating, and a concrete remediation step. A fully corrected version of the application (**secure_app.py**) is provided alongside this report.

CRITICAL	HIGH	MEDIUM	LOW	TOTAL
1	2	2	0	5

Figure 1: Vulnerability severity distribution

2. Scope and Methodology

Item	Details
Target File	vulnerable_app.py
Language / Runtime	Python 3.x
Application Type	CLI — Authentication + File Read
Analysis Method	Static Code Analysis, Manual Inspection
Tools Used	Manual review, Python interpreter, SQLite CLI
OWASP Reference	OWASP Top 10 2021 (A01–A09)

3. Vulnerability Findings

VUL-001 SQL Injection — Authentication Bypass		CRITICAL
Location	vulnerable_app.py — login() function, line ~43	
OWASP	A03:2021 – Injection	

Description

User-supplied input for username and password is concatenated directly into an SQL query string without sanitization or parameterization. An attacker can inject SQL syntax to bypass authentication entirely, extract all records from the database, or manipulate query logic.

Proof of Concept

```
Username: ' OR '1'='1' --
```

```
Password: anything
```

```
Resulting query:
```

```
SELECT * FROM users WHERE username='' OR '1'='1' --' AND password='...'
```

Impact

Complete authentication bypass. Any user can log in as any account, including admin, without knowing the password.

Remediation

Use parameterized queries (prepared statements). Pass user input as bind parameters, never via string concatenation.

```
cursor.execute(  
    "SELECT * FROM users WHERE username=? AND password_hash=?",  
    (username, hashed_input)  
)
```

VUL-002 Hardcoded Credentials in Source Code		HIGH
Location	vulnerable_app.py — module level, lines 17–18	
OWASP	A07:2021 – Identification and Authentication Failures	

Description

The admin username and plaintext password are hardcoded as module-level constants. Any developer, contractor, or attacker with access to the source file or version control history immediately obtains valid credentials.

Proof of Concept

```
ADMIN_USERNAME = "admin"
ADMIN_PASSWORD = "admin123" # visible to all
```

Impact

Immediate unauthorized administrative access. Credentials cannot be rotated without a code change and redeployment.

Remediation

Load credentials from environment variables or a secrets manager at runtime. Raise an error if they are absent rather than falling back to defaults.

```
ADMIN_USERNAME = os.environ.get("APP_ADMIN_USER")
ADMIN_PASSWORD = os.environ.get("APP_ADMIN_PASS")
if not ADMIN_USERNAME or not ADMIN_PASSWORD:
    raise EnvironmentError("Credentials not configured.")
```

VUL-003 Path Traversal — Arbitrary File Read		HIGH
Location	vulnerable_app.py — read_file() function, line ~65	
OWASP	A01:2021 – Broken Access Control	

Description

The read_file() function accepts a filename from user input and joins it with a base directory using os.path.join() without verifying that the resolved path stays within the intended directory. An attacker can escape the base directory using ../ sequences.

Proof of Concept

```
Enter filename to read: ../../etc/passwd
Resolves to: /etc/passwd
```

Impact

An authenticated user can read any file on the system that the process has permission to access, including /etc/passwd, SSH keys, and configuration files.

Remediation

Resolve the full real path with os.path.realpath() and verify it starts with the approved base directory before opening the file.

```
base = os.path.realpath(base_dir)
full = os.path.realpath(os.path.join(base, filename))
if not full.startswith(base + os.sep):
    print("[ERROR] Access denied.")
return None
```

VUL-004 Weak Password Hashing (MD5, No Salt)		MEDIUM
Location	vulnerable_app.py — init_db() and main(), lines ~28 and ~80	
OWASP	A02:2021 – Cryptographic Failures	

Description

Passwords are hashed using MD5 with no salt. MD5 is cryptographically broken: it is extremely fast to compute, has known collision vulnerabilities, and is thoroughly covered by precomputed rainbow tables. Identical passwords produce identical hashes, enabling bulk cracking.

Proof of Concept

```
MD5('admin123') = 0192023a7bbd73250516f069df18b500
# Instantly crackable via crackstation.net or hashcat
```

Impact

If the database is compromised, all password hashes can be reversed in minutes using freely available tools and rainbow tables.

Remediation

Use a slow, salted hashing algorithm. Minimum: SHA-256 with a per-user random salt. Recommended: bcrypt or argon2id via the passlib library.

```
import secrets, hashlib
salt = secrets.token_hex(16) # unique per user
hash = hashlib.sha256((salt + password).encode()).hexdigest()
# Store both salt and hash in the database
```

VUL-005 Insecure Error Handling — Information Leakage		MEDIUM
Location	vulnerable_app.py — login() and read_file(), exception blocks	
OWASP	A05:2021 – Security Misconfiguration	

Description

Exceptions are caught and the raw error message, full SQL query text, and internal file paths are printed directly to the user. This provides an attacker with a detailed map of the application's internal structure during reconnaissance and exploitation.

Proof of Concept

```
Login error output:
[ERROR] Database query failed: near "OR": syntax error
```

```
[DEBUG] Query was: SELECT * FROM users WHERE username='...' AND password='...'
```

Impact

Reveals database schema, query structure, file system paths, and error conditions that aid in crafting more targeted attacks.

Remediation

Log full error details to a secure server-side log file only. Show the user a generic, non-descriptive message that does not reveal internal state.

```
except Exception as e:
```

```
    logging.error("DB error: %s", e) # server-side log only
```

```
    print("[ERROR] An internal error occurred.") # generic user message
```

4. Remediation Summary

ID	Vulnerability	Severity	Remediation	Status
VUL-001	SQL Injection	CRITICAL	Parameterized queries	Fixed in <code>secure_app.py</code>
VUL-002	Hardcoded Credentials	HIGH	Environment variables	Fixed in <code>secure_app.py</code>
VUL-003	Path Traversal	HIGH	<code>realpath()</code> + prefix check	Fixed in <code>secure_app.py</code>
VUL-004	Weak Hashing (MD5)	MEDIUM	SHA-256 + unique salt	Fixed in <code>secure_app.py</code>
VUL-005	Information Leakage	MEDIUM	Generic errors + server log	Fixed in <code>secure_app.py</code>

5. Secure Coding Best Practices

Never trust user input

Validate, sanitize, and escape all input before using it in queries, file operations, or system calls.

Use parameterized queries always

Even for queries that appear safe, use prepared statements. It costs nothing and prevents injection entirely.

Store secrets in environment variables

Credentials, API keys, and tokens must never appear in source code or version control history.

Use strong, salted password hashing

Use `bcrypt` or `argon2id` for passwords. If using SHA-256, always generate a unique random salt per user.

Apply the principle of least privilege

File operations, database accounts, and process permissions should be scoped to the minimum required.

Fail securely and log internally

Show users only generic error messages. Write detailed diagnostics to a protected server-side log.

Enforce path confinement

Always resolve and verify file paths before opening them. Reject any path that escapes the intended directory.

Use static analysis tools

Integrate tools like `Bandit` (Python) into CI/CD pipelines to catch common security issues automatically.

6. Conclusion

The audit of `vulnerable_app.py` identified five distinct security vulnerabilities spanning injection, access control, cryptographic failures, and information disclosure — four of the OWASP Top 10 (2021) categories. All findings

have been remediated in the accompanying `secure_app.py` with explanatory inline comments.

The most critical issue — SQL Injection — allows complete authentication bypass with no credentials required, and must be treated as the highest priority fix in any production scenario. Hardcoded credentials and path traversal are equally dangerous and straightforward to exploit. Weak hashing and information leakage, while rated medium, significantly amplify the impact of other vulnerabilities if exploited in combination.

Adopting parameterized queries, environment-based configuration, strong password hashing, path confinement, and secure error handling across the codebase will eliminate all identified attack vectors and substantially raise the application's security posture.